

Institute of Mechatronic Systems (IMS)

Project thesis System Engineering

# Betaflight – Offline Controller Tuning

|                      |                                               |
|----------------------|-----------------------------------------------|
| Author(s)            | Yuri Bianchi<br>Janick Dort<br>Dario Jurietti |
| Main supervisor      | Michael Ernst Peter                           |
| Secondary supervisor | Ruprecht Altenburger                          |
| Date                 | 19.12.2025                                    |

## 1 Abstract

This project focuses on the development of a tool for offline tuning of gyro controllers for FPV racing drones based on the open-source firmware Betaflight. The quality of the gyro control loop has a direct influence on flight stability, the suppression of disturbances such as prop wash, and the agility of the drone. Until now, rate controller tuning has mostly been performed through time-consuming trial and error.

The aim of the project is to develop a modular, open-source solution for offline tuning based on an existing proof of concept. The code will be implemented in MATLAB, accompanied by a lightweight Python version, and supplemented with detailed Markdown documentation and examples on GitHub. The project aims to provide the community with a simple way to efficiently optimise drone parameters and thereby improve flight performance.

The results will be published in a public GitHub repository and documented in a report. In addition to the software development, the project also includes practical work with FPV drones to validate the developed methods.

## 2 Foreword

This document summarizes the results of our project thesis, which we worked on during the fall semester 2025 at the ZHAW School of Engineering. For us as systems engineering students, this project provided an ideal opportunity to apply advanced control theory and signal processing methods to a real, highly dynamic system. FPV drones present an interesting engineering challenge due to their highly dynamic behaviour and the need for precise control. This motivated us to investigate mathematical modelling, control engineering aspects, and offline tuning methods aimed at improving flight performance.

Our motivation was to translate the theoretical control concepts we learned over the past three years to a real, highly dynamic system. We also aimed to develop a tool that not only meets engineering standards but also provides real value to the FPV community by replacing subjective tuning with objective data analysis.

This work was carried out at the Institute of Mechatronic Systems (IMS) at the ZHAW School of Engineering. We would like to express our gratitude to our supervisor, Michael Peter, for his guidance and support throughout this project. His expertise, constructive feedback, and willingness to take the time to assist us in developing complex algorithms were invaluable.

## Inhaltsverzeichnis

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Abstract</b>                                                   | <b>2</b>  |
| <b>2</b> | <b>Foreword</b>                                                   | <b>2</b>  |
| <b>3</b> | <b>Introduction</b>                                               | <b>5</b>  |
| 3.1      | Initial situation . . . . .                                       | 5         |
| 3.2      | Task and Objective . . . . .                                      | 5         |
| <b>4</b> | <b>Preparation</b>                                                | <b>7</b>  |
| 4.1      | Chirp . . . . .                                                   | 7         |
| 4.1.1    | Chirp Concept . . . . .                                           | 7         |
| 4.1.2    | Chirp on a drone . . . . .                                        | 7         |
| 4.1.3    | Chirp programming . . . . .                                       | 8         |
| <b>5</b> | <b>Theory</b>                                                     | <b>9</b>  |
| 5.1      | Signal Processing . . . . .                                       | 9         |
| 5.1.1    | Data Import . . . . .                                             | 9         |
| 5.1.2    | Handle High-Resolution Data . . . . .                             | 9         |
| 5.1.3    | Time Conversion and Sampling Rate Verification . . . . .          | 9         |
| 5.1.4    | Application of Rotational Filtering (Apply Rotfiltfilt) . . . . . | 10        |
| 5.1.5    | Estimation of Frequency Response . . . . .                        | 10        |
| 5.2      | Control System . . . . .                                          | 11        |
| 5.2.1    | Transfer Function Estimation . . . . .                            | 11        |
| 5.2.2    | Closed Loop Calculation . . . . .                                 | 11        |
| 5.2.3    | Filters and Controllers . . . . .                                 | 12        |
| 5.2.4    | Compensation of the I-Term . . . . .                              | 12        |
| 5.3      | Evaluation . . . . .                                              | 13        |
| 5.3.1    | Step Response . . . . .                                           | 13        |
| 5.3.2    | Spectra Analysis . . . . .                                        | 13        |
| 5.3.3    | Spectrum Analysis . . . . .                                       | 13        |
| 5.3.4    | Gang of Four . . . . .                                            | 13        |
| <b>6</b> | <b>Challenges and Methodological Approach</b>                     | <b>14</b> |
| 6.1      | Simulations . . . . .                                             | 14        |
| 6.1.1    | Simulation of Betaflight Controller Order . . . . .               | 14        |
| 6.1.2    | Simulation PID Controller . . . . .                               | 14        |
| 6.1.3    | Simulation Frequency Response . . . . .                           | 14        |
| 6.1.4    | Simulation Spectra . . . . .                                      | 15        |
| 6.1.5    | Simulation Spectrogram . . . . .                                  | 15        |
| 6.2      | Building an FPV drone . . . . .                                   | 16        |
| 6.3      | Tuning an FPV drone . . . . .                                     | 16        |
| 6.4      | Decode . . . . .                                                  | 18        |
| 6.5      | Code Structure and programming style . . . . .                    | 19        |
| 6.5.1    | OOP vs. Functional Approach . . . . .                             | 19        |
| 6.5.2    | Final Structure Concept . . . . .                                 | 21        |
| 6.6      | Conversion MATLAB to Python . . . . .                             | 23        |

|           |                                             |           |
|-----------|---------------------------------------------|-----------|
| 6.7       | Python in MATLAB . . . . .                  | 23        |
| 6.8       | Transfer MATLAB Library to Python . . . . . | 23        |
| <b>7</b>  | <b>Results</b>                              | <b>24</b> |
| <b>8</b>  | <b>Discussion and next steps</b>            | <b>25</b> |
| 8.1       | Discussion . . . . .                        | 25        |
| 8.2       | Next Steps . . . . .                        | 25        |
| 8.3       | Conclusion . . . . .                        | 25        |
| <b>9</b>  | <b>Directories</b>                          | <b>27</b> |
| 9.1       | Bibliography . . . . .                      | 27        |
| 9.1.1     | Overview of Generative AI Systems . . . . . | 27        |
| 9.2       | List of Figures . . . . .                   | 28        |
| <b>10</b> | <b>Appendix</b>                             | <b>30</b> |
| 10.1      | Project Management . . . . .                | 30        |
| 10.1.1    | Project Assignment . . . . .                | 30        |
| 10.1.2    | Project Schedule – Version 0 . . . . .      | 33        |
| 10.1.3    | Project Schedule – Version 1 . . . . .      | 33        |
| 10.1.4    | Project Schedule – Version 2 . . . . .      | 34        |
| 10.2      | Additional Information . . . . .            | 35        |

### 3 Introduction

First-person-view (FPV) racing drones require precise and responsive control to achieve high flight stability, minimize disturbances such as prop wash, and maintain agility during rapid manoeuvres. Proper tuning of the rate controllers is key to this performance, as it directly determines how the drone reacts to pilot inputs and external influences. Until now, tuning these controllers has mainly relied on iterative trial-and-error methods, a process that is both time-consuming and difficult to perform, especially for less experienced pilots.

Betaflight, an open-source firmware widely used in the FPV community, provides a flexible platform for configuring flight controllers. However, despite its popularity, efficient tuning remains a challenge. To address this, our project builds upon an existing proof of concept for offline tuning of rate controllers using flight data and system identification techniques. The goal is to transform this concept into a robust, modular, and open-source library that simplifies the tuning process.

The library will be implemented in 'MATLAB and Python', offering tools for analyzing flight logs, estimating system dynamics, and optimizing controller parameters without requiring repeated flight tests. A documentation is created, which shows how to use the tool and will be published on GitHub to ensure accessibility for the FPV community.

Besides software development, the project involves also building an FPV drone and testing and confirm the effectiveness of the proposed methods. By providing a structured, data-driven approach to tuning, this work aims to improve flight performance and reduce the barriers to achieving optimal control settings for both hobbyists and professionals.

#### 3.1 Initial situation

The open-source firmware Betaflight is the most widely used platform for FPV racing drones and is continuously developed by a large community. The quality of the rate control is crucial for flight performance, as it enables stability, agility, and effective suppression of disturbances such as prop wash.

Today, tuning of controller parameters is still largely performed by trial-and-error adjustments, a method that is time-consuming and often challenging for less experienced pilots. A proof of concept (developed by Michael Peter) already exists, which enables offline tuning of rate controllers using flight measurement data obtained from chirp signal excitation. However, the current implementation is based on MATLAB, which is not widely used within the FPV community and therefore limits accessibility. In addition, the proof-of-concept requires a certain level of technical knowledge to operate, which further complicates its practical use for many pilots.

#### 3.2 Task and Objective

The aim of this project is to further develop the existing proof of concept into an accessible and practical tool for tuning rate controllers on Betaflight-based FPV drones. The focus lies on transforming the prototype from a research-oriented MATLAB implementation into a solution that can be used by the wider community with minimal technical barriers. This includes designing a clean and modular software architecture, improving usability, and integrating functions for importing, processing, and analysing flight data.

In addition, the tool will be accompanied by comprehensive Markdown documentation, including explanations of key methods, usage instructions, and practical examples to support new users. The implementation will serve as a foundation for future developments, such as extended analysis features, alternative controller designs, or deployment in other embedded flight control systems. The overarching objective is to provide a reliable, data-driven workflow that replaces subjective trial-and-error tuning and enables verifiable control performance improvements.

## 4 Preparation

### 4.1 Chirp

#### 4.1.1 Chirp Concept

A chirp is an input signal that sweeps continuously over a defined range (e.g. from a low frequency to a high frequency). This signal excites the system on multiple frequencies in one run and thus lets you observe the frequency response and step response, the foundation of controller tuning. Mathematically, a chirp signal can be represented as a sinusoidal wave with instantaneous frequency that varies linearly or exponentially with time. For more information about the mathematical background, please have a look in the document [Chirp Signal](#).

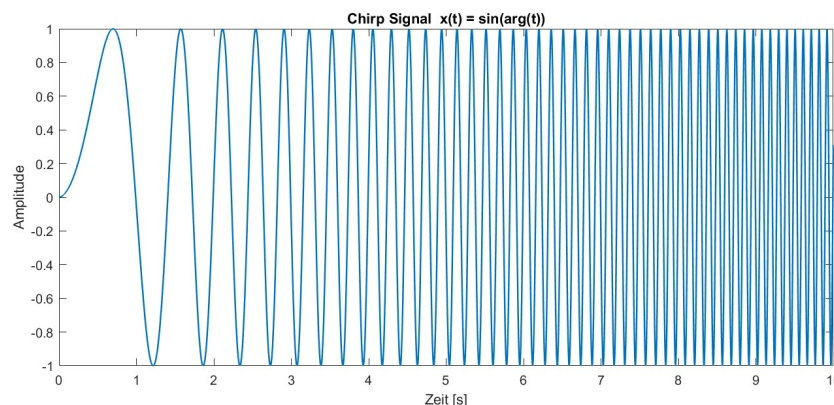


Abbildung 1: Chirp Signal example

#### 4.1.2 Chirp on a drone

In the context of tuning a drone's flight controller, an exponential sweep is used. This is due to the dominant low-frequency dynamics, where low frequencies exhibit more predictable responses, allowing the sweep to allocate more time to higher frequencies for better resolution of system behavior. Additionally, multirotor vibration resonances are typically spaced logarithmically, making exponential sweeps more suited for efficient excitation across the frequency range.

The process involves these steps:

1. **Signal Injection:** The chirp signal is added to the pilot's stick input. This composite signal becomes the setpoint for the PID controller.
2. **System Response:** The drone attempts to follow this rapidly changing setpoint. The motors react, and the drone will oscillate at varying frequencies corresponding to the chirp signal.
3. **Data Logging:** While the chirp is active, high-resolution data from the gyroscope and the motor outputs are recorded using Betaflight's Blackbox feature. This data captures how the drone responds to the control inputs at different frequencies.

The recorded log file, containing the input chirp and the corresponding gyroscope output, is then used for offline analysis to determine the frequency response of the system.

### 4.1.3 Chirp programming

The implementation of the chirp signal in Betaflight is straightforward. You call the function `chirpInit()` and pass:

- a struct of type `chirp_t` defined in `chirp.h`
- starting frequency
- end frequency
- signal length
- loop-time in microseconds

This initialises the values in the data struct and:

- converts loop period from microseconds to seconds
- calculates the number of samples in the signal duration
- calculates the exponential growth factor
- precomputes the constants used to get the phase from the frequency evolution
- calls `chirpReset()` function

The `chirpReset()` function sets:

- internal counter to 0
- bool `isFinished` to false

and then calls the function `chirpResetSignals()` to set signal-related fields to 0.

Once the chirp is initialised use the `chirpUpdate()` function once per loop iteration.

This function checks:

- if bool `isFinished` is true -> return false
- else if internal counter is equal to the number of samples inside the chirps period  
-> set bool `isFinished` true, call `chirpResetSignals()`, return false

if none of the above is true the ...



## 5 Theory

### 5.1 Signal Processing

Signal processing applied to Betaflight Blackbox logs transforms raw flight data into usable information for in-depth analysis. These logs contain flight measurements such as stick inputs, gyro signals, PID responses and more. Although the dataset may appear complex at first, software solutions including our analysis tool and the [Betaflight Blackbox Explorer](#) make interpretation more straightforward by providing visualisations and metrics that support troubleshooting, tuning and performance evaluation.

Preparing the logged data is a central part of the workflow. Before any analysis can be performed, the raw signals are conditioned into a consistent and meaningful representation. This typically begins with filtering to suppress noise without altering the underlying dynamics. Based on the processed signals, the frequency response of the control system can be estimated to characterise how it reacts to different input spectra. Each processing step increases the reliability of the data and leads to more robust conclusions in subsequent analysis.

#### 5.1.1 Data Import

The first step of any analysis is to load the measurement data properly. Betaflight Blackbox logs consist of two essential parts: the header and the data block. The header contains key information about the conditions under which the flight data was recorded, including loop rates, logging settings, active filter configurations and the PID controller values. Without this information, calculations cannot be performed correctly and the recorded data loses its analytical value.

Data imports are unfortunately very computationally expensive. Therefore, to save time during analysis, the logs are converted from raw CSV files into a faster '.mat' format. This way, you don't need to reprocess the large log files every time. The signal names (like gyro and motor outputs) are also automatically assigned to their correct data columns. This makes it easy to find and assign the signals later. For more details, check the [Dataimport Documentation](#).

#### 5.1.2 Handle High-Resolution Data

Betaflight provides a High Resolution Mode as an optional logging setting, in which selected signals are multiplied by a factor of ten before being written to the log. Because these values are stored as integers, this reduces decimal precision requirements and minimises rounding effects. During analysis, the affected signals must be scaled back to their original units; otherwise, frequency plots, controller tuning results or general interpretations will be incorrect. Important signals such as gyro data and control inputs are affected by this step. To get more information, check the [Unscale high Resolution Gain documentation](#).

#### 5.1.3 Time Conversion and Sampling Rate Verification

In a Betaflight log, time is stored as an increasing microsecond counter rather than as regular seconds. Because of hardware limits or system load, the spacing between time stamps isn't always the same. This step converts the time information into a uniform time base, ensuring that all samples can be compared consistently and interpreted in their correct temporal context.

By checking the time differences between samples, you can detect logging issues such as dropped frames or small variations in the actual sampling rate. Correct handling of the time information is important for all further analysis steps, for example frequency calculations or controller tuning. For a more detailed explanation, see the [Convert and evolution Time documentation](#). If you are interested in tuning your drone, you can also refer to the [Evolution Time Flight description](#).

#### 5.1.4 Application of Rotational Filtering (Apply Rotfiltfilt)

Filtering is a fundamental step in preparing the data for analysis, as it removes unwanted noise while preserving the original characteristics of the signals. The function `Apply Rotfiltfilt` operation applies a zero-phase filter, ensuring that the data's phase relationships remain intact while suppressing selected frequency components such as low-frequency drifts or high-frequency noise.

This step applies a forward-backward filtering technique, which processes the data in both forward and reverse directions and effectively cancels out phase distortions. This method is particularly useful when working with chirp signals, as it enables precise filtering without introducing phase shifts that could alter the signal's characteristics.

Proper application of this technique ensures that the data is clean and reliable for subsequent analysis steps. For more insights on how this method works, refer to the full description in the [Apply Rotfiltfilt documentation](#).

#### 5.1.5 Estimation of Frequency Response

The `Estimate Frequency Response` function uses advanced techniques to measure and analyze the relationship between a system's input and output signals in the frequency domain. This makes it possible to determine how different frequencies are amplified or attenuated by the system, providing valuable information about its dynamic behavior.

This technique is based on the Welch Method, which divides the signal into overlapping segments. By applying a Hann window to each segment and performing spectral averaging, the influence of noise is reduced and a smoother estimate of the frequency response is obtained. The result is a single-sided, amplitude-calibrated response curve that shows how individual frequencies are amplified or attenuated, as well as how phase shifts occur across the spectrum.

The resulting data is not only accurate but also robust, making it suitable for applications like control system tuning or stability analysis. For a deeper dive into the theoretical principles and implementation steps, consult the detailed [Estimate Frequency Response documentation](#).

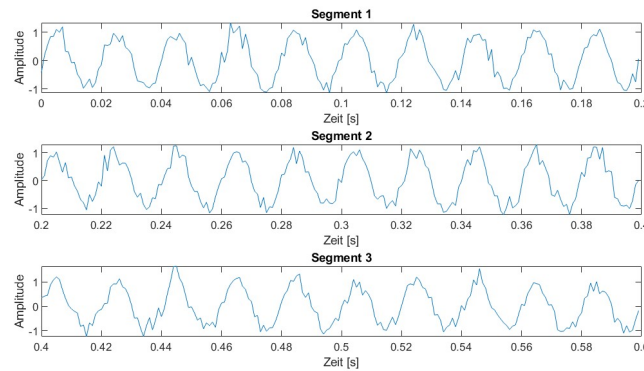


Abbildung 2: Segmentations of Data

## 5.2 Control System

The control system determines the dynamic response of a drone and is therefore central to flight stability and precise maneuvering. In this section, we analyse the individual components of the controller, examine their influence on the system behaviour and outline the methods used for tuning in order to improve performance and responsiveness.

### 5.2.1 Transfer Function Estimation

To analyse how the system reacts to control inputs, the transfer function of the plant is estimated directly from closed-loop flight data. This makes it possible to characterise the system dynamics without opening the control loop and to determine how input signals are transformed into outputs. The resulting model forms the basis for matching the controller parameters to the real system. More details can be found in the [Estimate Transfer Function documentation](#).

### 5.2.2 Closed Loop Calculation

The interaction between the controller and the estimated transfer function of the plant determines the stability and robustness of the closed-loop system. By calculating the closed-loop transfer functions for both inner and outer loops, sensitivity, disturbance rejection, and overall system stability can be evaluated. These calculations provide a clear view of the controller dynamics and their impact on the system response. For details on the implementation, see the [Calculate Closed Loop documentation](#).

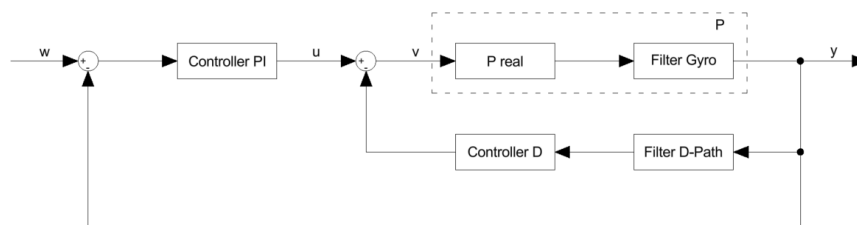


Abbildung 3: Gyro Control Loop

### 5.2.3 Filters and Controllers

Filters and controllers form the foundation of the control loop. Filters remove unwanted noise from sensor data or motor outputs, while controllers ensure that the flight system remains stable and tracks the desired setpoints accurately. The system makes use of various low-pass, notch, and dynamic filters that are designed to optimize the control process. The controllers, such as the PI and D components, are then tuned to achieve the desired balance between reactivity and stability. For further insights, see the [Filters and Controllers documentation](#).

### 5.2.4 Compensation of the I-Term

The integral term in a PID controller addresses long-term errors, ensuring steady corrections over time. However, it can become problematic during quick stick movements, where it may overcorrect and destabilize the system. To avoid this, compensation methods adjust the influence of the I-term based on the situation, suppressing it during rapid changes while allowing it to integrate errors during slower movements. This approach helps maintain both precision and responsiveness. Have a look at [Compensate I-Term documentation](#) to see how we added compensation to the calculation.

## 5.3 Evaluation

Evaluation is an essential part of understanding and improving the behaviour of the control system. This section outlines the key analytical steps, such as analysing time-domain responses and interpreting frequency-domain data to evaluate the system dynamics. Detailed information on how to interpret the generated figures can be found in the [Descriptions Figures](#).

### 5.3.1 Step Response

The step response provides a direct view of how the system reacts to sudden changes in the input signal. By transforming the closed-loop transfer function from the frequency domain into the time domain, the dynamic behaviour of the control loop can be evaluated. Key performance indicators such as rise time, overshoot, settling time, and steady-state error can be determined to assess system stability and responsiveness. The calculations use inverse Fourier transforms of frequency response data to reconstruct the system's step response curve. This curve can be used to identify underdamped or insufficiently tuned dynamics. More details can be found in the [Step Response documentation](#).

### 5.3.2 Spectra Analysis

Spectral analysis examines how the system responds to different input frequencies and provides valuable insight into its dynamic characteristics. The method separates complex signals into their frequency components using algorithms such as Welch's method. Overlapping signal segments, combined with windowing techniques like the Hann window, yield smoother and statistically more reliable estimates of power spectra. This frequency-domain perspective highlights resonances, noise characteristics, and the effectiveness of applied filters. The steps of this process are described in detail in the [Spectra Analysis documentation](#).

### 5.3.3 Spectrum Analysis

Spectrum analysis builds on spectral analysis by providing a multi-dimensional view of the signal's frequency content over time or, in this case, thrust. By grouping FFT segments into bins along an additional axis, such as throttle or angle, the power spectrum can be mapped as a function of both frequency and the selected variable. This approach is particularly useful in applications that require an understanding of how system dynamics change during flight. Relevant methodologies and results are described in the [Spectrum Analysis section](#).

### 5.3.4 Gang of Four

New!!!

## 6 Challenges and Methodological Approach

### 6.1 Simulations

At the beginning of our thesis, we started creating simulations. We used these simulations to deepen our understanding of the existing code and the methods it employed. Although the simulations were time-intensive, they proved crucial for our work. They formed the basis for all subsequent analysis steps and helped us understand the existing functions before applying them further.

#### 6.1.1 Simulation of Betaflight Controller Order

To analyse and better understand the Betaflight control system, we recreated the complete control structure in MATLAB. The corresponding script can be found in the attachments. In this simulation, we attempted to reproduce the step response of the recorded data by using the same PID parameters and filter settings as in the real system. The different filters were created according to the documentation [Filter and Controller](#). We implemented the filters both in continuous and discrete form, allowing us to compare the two approaches. The PI and D controllers were built in the same way. For the plant transfer function, we used a simplified **Proportional Time constant element (PT1)** combined with dead time, based on the approaches described in the documentation [Calculation Closed Loop](#). In addition to the MATLAB code, we created a Simulink model to test the step response of our Betaflight control system simulation. The simulated response closely matched the step response of the real drone, confirming that we had correctly modelled the controller order.

#### 6.1.2 Simulation PID Controller

To deepen our understanding of the different controller components, we created an additional MATLAB script. This script allowed us to combine various controller types with simple plant models. The user can switch between P, I, PI, PD and PID controllers, as well as between basic plant types. This enabled us to observe how each controller component influences the overall system behaviour. The script calculates the open-loop and closed-loop transfer functions for the chosen settings and generates Bode plots, Nyquist diagrams, and step responses. This helped us understand and visualise the effect of each component, significantly improving our overall understanding of controller behaviour.

#### 6.1.3 Simulation Frequency Response

To analyse the frequency response of a system based on input and output data, we created a dedicated simulation. In this simulation, we generated a chirp signal with added noise to replicate realistic measurement conditions. Using these signals, we implemented different methods to estimate the system's transfer function. First, we calculated a basic frequency response using the plain **Fast Fourier Transformation (FFT)**, which served as a benchmark. After that, we applied various signal processing methods. We began with the Welch method to improve the estimation by averaging over overlapping segments. The implementation of the Welch **Frequency Response Function (FRF)** follows the theory described in the documentation [Estimate Frequency Response](#). This allowed us to compare the plain FFT approach with the more robust Welch estimator. In the next step, we implemented a rotated-signal approach, where the chirp is demodulated to baseband before filtering. For this part, we used the zero-phase filtering method `apply_rotfiltfilt`, as described in the documentation [Apply Rotfiltfilt](#). We then compared the original Bode plot to the processed one

to evaluate how the rotation technique improves noise robustness. Overall, this script helped us understand how noise affects the estimated transfer function and how signal processing techniques can increase robustness.

#### 6.1.4 Simulation Spectra

We developed a simulation to examine the different components used to create a meaningful spectrum, based on the system used in the existing code. In this script, we generated a noisy sinusoidal input signal from which we created different spectra. In one case, we applied a window function, while in the other we used no window at all. By splitting the signal into several overlapping segments, we observed how averaging across segments reduces noise and stabilises the resulting spectrum. The implementation is described in detail in the document [Spectra Analysis](#). This exercise helped us understand how windowing, overlap, and segment length affect spectral estimation, as well as how these methods can increase robustness and reduce spectral leakage.

#### 6.1.5 Simulation Spectrogram

To analyse how the frequency content of the excitation signal changes with both time and thrust, we developed an additional MATLAB simulation. In this script, we generated a noisy sinusoidal input signal and a slowly varying thrust signal. The input signal was then divided into segments and windowed with a Hann window. Each segment was transformed using the FFT to obtain a time-frequency representation. First, we arranged the segment spectra along the time axis, and in a second step, along the thrust levels. This resulted in two spectrograms: one showing amplitude over frequency and time, and the other showing amplitude over frequency and thrust. Through this visualisation, we could identify the dominant frequency components as a function of thrust. The underlying method is described in detail in the document [Spectrogram Analysis](#).



## 6.2 Building an FPV drone

Besides the understanding of the code and further developing the proof of concept, building a drone and set it up correctly bridges the gap between the digital domain, where algorithms and simulations are developed and the physical reality of drone operation. By getting in touch with the hardware, it is easier for us to understand what practical challenges exist for pilots and the community.



Abbildung 4: IFLIGHT aosmini build

Therefore a three inch FPV drone was built from the ground up. This work included:

- creating schemes to see which connections were required
- soldering all the components as the motors, receiver, video system, logger and gps module to the flight controller
- attaching everything to the frame, so that nothing generates unnecessary oscillations during flight
- double check all soldering connections and screws
- setting everything up in betafly
- testfly the drone

The biggest challenge was to achieve good soldering connections ...

## 6.3 Tuning an FPV drone

As part of understanding the workflow we tuned an FPV drone by ourselves. The goal was to adjust the PID parameters so that we achieve a fast rising step response without overshoot, a lower compliance than before and a rounded shaped Sensitivity. Also to mention is, that the axes roll



and pitch should be tuned as similar to each other as possible. This is because you want the drone to maintain a consistent and predictable response, so roll and pitch behave similarly for smoother control and a balanced flight experience.

Initially, we deliberately applied a poor tuning configuration to observe its effects. During the first field test with this setup, it quickly became apparent that the tuning was excessively poor. The drone began oscillating aggressively and climbed rapidly. As the drone was out of control, the kill switch had to be activated, resulting in a crash that broke one of the four arms. Fortunately, the damage was minor and repaired quickly.

After this incident, we decided not to repeat flights with intentionally bad tuning. Instead, we continued using the default Betaflight parameters as a baseline. **With these the drone flies reliably.** However, during demanding manoeuvres such as flips or propwash manoeuvres, it becomes evident that there is room for improvement in terms of stability and responsiveness.

The tuning process was carried out on a larger 6-inch FPV drone provided by one of the team members. The drone was freshly flashed with Betaflight v12, meaning all filters and settings were at factory defaults. Achieving an optimal tune required several attempts due to challenges with filter adjustments and occasional logging issues. Nevertheless, the drone was successfully tuned, and the improvement was noticeable. During propwash manoeuvres, the drone remained significantly more stable, and flips were executed with greater precision at the stop.

Our tuning objective was to achieve a fast step response without overshoot, while maintaining good compliance and sensitivity. It is important to note that there is no universally “perfect” tune. Settings depend heavily on pilot preferences. Racing pilots, for example, may want more responsiveness for competitive performance, whereas freestyle pilots want smoothness and flight feel.

Since none of our team members are professional pilots, we focused on achieving robustness and a step response without overshoot. This tuning approach is probably somewhere between what a racing pilot and a freestyle pilot would prefer. Robustness in this context is important for practical scenarios, such as when a propeller is slightly bent, the battery sits slightly off center or when an additional component like a camera is mounted on the drone.

## 6.4 Decode

Text...

## 6.5 Code Structure and programming style

From the start of the project, we recognized that a clear and maintainable code structure would be essential. The initial proof-of-concept code already contained a rough separation into topics such as data I/O and step response, but everything was packed into a single file, making it difficult to navigate. Our first step was to comment out the proof-of-concept code and reorganize it into thematic sections. This was the foundation for a more organized layout and we came to an initial directory structure:

```
bf_controller_tuning/  
├── main.m  
├── data_io/  
│   ├── load_data.m  
│   ├── extract_header_information.m  
│   └── preprocess_data.m  
├── flight_parameters/  
│   ├── get_pid_parameters.m  
│   ├── scale_pid_parameters.m  
│   └── print_parameters.m  
├── spectrogram/  
│   ├── calculate_spectrogram.m  
│   ├── plot_spectrogram.m  
│   └── show_spectrogram.m  
├── spectra/  
│   ├── estimate_spectra.m  
│   ├── plot_spectra.m  
│   └── show_spectra.m  
├── frequency_response/  
│   ├── estimate_frequency_response.m  
│   ├── plot_bode.m  
│   └── show_frequency_response.m  
├── controller_analysis/  
│   ├── calculate_closed_loop.m  
│   ├── calculate_step_response.m  
│   └── show_controller_analysis.m  
└── utils/  
    ├── apply_rotfiltfilt.m  
    ├── downsample_frd.m  
    └── get_my_colors.m
```

This draft was never intended to be final. Rather, it served as an initial attempt to bring clarity and separation between different functional areas.

### 6.5.1 OOP vs. Functional Approach

At this stage, we discussed whether to implement the entire project using object-oriented programming (OOP) or stick to a modular, function-based approach. Initially, we considered OOP as a challenge, since most of us were not very experienced with it. We started writing some components

as classes, but quickly realized that only one team member was really comfortable with OOP, while the others struggled. This made us reconsider. Our conclusion was to use OOP selectively—only for parts that benefit from reuse and encapsulation, such as data loading and plotting, while keeping the rest function-based for simplicity. This hybrid approach allowed us to maintain flexibility without overcomplicating the design.

After several attempts, we agreed on the following principles:

- A main script (main.m) for user input, figure selection, filter settings, and PID tuning
- A core class to handle all calculations (essentially the logic from the proof of concept, but without plotting)
- A plot utilities class to centralize all plotting functions for consistency

Existing library functions from the proof of concept were reused without modification. The thematic grouping included:

```

bf_controller_tuning/
├── class/
│   ├── main_class.m
│   └── plot_utils.m
├── lib/
│   ├── apply_rotfiltfilt.m
│   ├── calculate_closed_loop.m
│   ├── calculate_controllers.m
│   ├── calculate_step_response_from_frd.m
│   ├── calculate_transfer_functions.m
│   ├── downsample_frd.m
│   ├── estimate_frequency_response.m
│   ├── estimate_spectra.m
│   ├── estimate_spectrogram.m
│   ├── expand_multiple_figure_nr.m
│   ├── extract_header_information.m
│   ├── get_chirp_signals.m
│   ├── get_fcut_from_D_and_fcenter.m
│   ├── get_fcut_from_exp.m
│   ├── get_filter.m
│   ├── get_ind_eval.m
│   ├── get_my_colors.m
│   ├── get_notch_Q.m
│   ├── get_pid_scale.m
│   └── get_switch_case_text_from_para.m
├── logs/
└── main.m
  
```

This structure worked well and kept the main script simple for the user. However, after a review meeting, our instructor suggested further modularization. The reason: we had created a “god class,” which limited flexibility. For example, if someone wanted to use the tool only for plotting spectrograms without any drone-specific logic, the current design made that difficult. Also the

program was slow. All calculations were made, every time the user changed something. We wanted to improve the speed in only doing these calculations again, which are relevant on that specific user input change.

### 6.5.2 Final Structure Concept

Out of this, we decided to split up this god class into three classes. The `flight_analyzer` class handles **frequency domain analysis and vibration characterization**. Its core responsibility is performing spectral analysis on flight log data to identify resonance frequencies and assess filter effectiveness. It computes power spectral density of gyro signals (both filtered and unfiltered) and generates spectrograms showing how vibration frequencies vary with throttle levels across all three axes (roll, pitch, yaw). This analysis is essential for identifying noise sources, evaluating filtering strategies, and optimizing drone stability during tuning. The `flight_data` class is responsible for **loading, parsing, and managing flight log data**. It provides methods to extract relevant information from Blackbox logs, such as gyro and motor data, and prepares this data for analysis. The `gyro_ctrl_tuning` class focuses on **tuning the PID controllers** for the drone's flight. It uses the results from the frequency response and spectral analysis to adjust the PID parameters, aiming to optimize the drone's flight characteristics. The `plot_utils` class remains responsible for **visualization and graphical representation** of all analysis results. Unlike the previous approach, this class is now more flexible and reusable. It can be used independently to generate plots from pre-calculated data, making it suitable for users who only want to visualize spectrograms, frequency responses, or step responses without running the full analysis pipeline. This modularity allows the plotting functionality to be easily adapted for different visualization needs and integrated into other workflows.

This approach still holds thematic sections, but is more accessible to use only specific parts of the tool.

```

bf_controller_tuning/
├── class/
│   ├── flight_analyzer.m
│   ├── flight_data.m
│   ├── gyro_ctrl_tuning.m
│   └── plot_utils.m
└── lib/
    ├── apply_rotfiltfilt.m
    ├── calculate_closed_loop.m
    ├── calculate_controllers.m
    ├── calculate_step_response_from_frd.m
    ├── calculate_transfer_functions.m
    ├── downsample_frd.m
    ├── estimate_frequency_response.m
    ├── estimate_spectra.m
    ├── estimate_spectrogram.m
    ├── expand_multiple_figure_nr.m
    ├── extract_header_information.m
    ├── get_chirp_signals.m
    ├── get_fcut_from_D_and_fcenter.m
    └── get_fcut_from_exp.m
  
```

```
├── get_filter.m
├── get_ind_eval.m
├── get_my_colors.m
├── get_notch_Q.m
├── get_pid_scale.m
├── get_switch_case_text_from_para.m
├── logs/
└── main.m
```

## **6.6 Conversion MATLAB to Python**

Text...

## **6.7 Python in MATLAB**

Text...

## **6.8 Transfer MATLAB Library to Python**

Text...

## 7 Results

Throughout the project, we achieved a range of practical and technical results. The most important outcome was gaining a deep understanding of the Betaflight control system, including how its filter structures and controller components interact to stabilise the gyro control loop. In addition, we developed a solid foundation in the application of essential signal-processing methods and learned how to use them correctly for frequency-domain analysis, step-response estimation and data preparation. This newly acquired knowledge will be crucial for the further development of the tool and forms an important basis for the work that will follow in the bachelor's thesis. The simulations carried out during the project helped us link theoretical concepts to the real dynamic behaviour of an FPV drone and validate our understanding of the control loops.

Alongside the theoretical work, we also built and configured an FPV drone from scratch and performed flight tests and tuning. This hands-on experience provided valuable insight into the practical challenges of drone construction, setup, flight and tuning. It helped us better understand the real-world implications of our analysis, the needs of the FPV community, and ultimately strengthened the connection between theory and practice.

On the software side, we restructured the existing prototype into a modular codebase, created comprehensive Markdown documentation on GitHub, and began transferring key functions from MATLAB to Python to enable future extensions. We also gained experience in object-oriented programming and code organisation for larger projects. In addition, we made significant progress in migrating core functions from MATLAB to Python, such as closed-loop calculations and transfer-function estimation. Although the full transfer is still ongoing, the results achieved so far provide a strong foundation for further development and refinement during the bachelor's thesis.

A particularly time-consuming part of the project was the development of an automatic extraction method for flight data directly from Betaflight Blackbox logs. Despite testing different approaches, we were not able to complete a reliable solution within the given time frame. However, the insights gained during this process form a valuable basis for further investigations, and we plan to revisit this topic as part of the bachelor's thesis.

In summary, the project demonstrates that offline analysis and tuning of Betaflight-based FPV drones provides a robust, objective and data-driven approach to optimising flight performance. The implemented methods enable the reconstruction of control-loop behaviour from flight measurements, allowing for a detailed understanding of system dynamics and the impact of controller settings. The work carried out so far lays a solid foundation for the bachelor's thesis, during which the tool will be extended with additional tuning capabilities, fully migrated to Python, and validated through further flight tests.



## 8 Discussion and next steps

### 8.1 Discussion

Overall, the work in the last 14 weeks mainly served as a foundation for the future work at the project during our Bachelor thesis rather than a finished end product. We were able to lay the groundwork for further developments. For example, we newly structured the code, which allows us to make additions and upgrades in the code faster and clearer. But more importantly, we were able to learn much about the behaviour of the Betaflight control system and how we have to face challenges in it. Dario Juriatti and Janick Dort also learned how a drone flies and how to build an FPV drone. This experience helped them to better relate the abstract analysis results and behaviour in the air.

Additionally to this, we deepened our understanding how advanced signal-processing methods and more complex control loops work. We achieved this through a series of simulations which made it easier to connect the theory to the measured data of real flights. Also, we improved our skills in object-oriented programming and how to structure a big code. We also were able to get our first experience in using control-engineering in Python, even though the Python implementation is not finished yet. All of this will help us to develop new features quickly and with high quality in our Bachelor thesis.

### 8.2 Next Steps

This project is being continued and further developed as part of a bachelor's thesis. In this context, we plan to extend the existing codebase with new features such as offline tuning for position and altitude hold. These additions will provide deeper insight into the behaviour of the corresponding control loops and allow their optimisation.

Furthermore, a full transfer of the implementation from MATLAB to Python is planned. This will make the tool more accessible to the FPV community, as Python is open source, widely used and free of charge. Our long-term goal is to enable seamless integration of the tool into existing Betaflight utilities, so that pilots can use it for offline tuning of their drones.

Finally, we intend to validate the analysis results through additional flight tests. By comparing the outcomes of the offline analysis with real flight behaviour, we aim to ensure that the methods are accurate and reliable. This will help us further refine the tool and increase its practical value for the FPV community.

### 8.3 Conclusion

Throughout the project, we gained a deep understanding of the Betaflight control system, learned how its filter and controller structures interact, and applied advanced signal-processing techniques for frequency-domain analysis, step-response evaluation and flight-data preparation. Simulations allowed us to connect theoretical models with observed flight behaviour, while building and test-flying an FPV drone helped us translate analytical findings into practical insights. On the software side, we restructured the existing prototype into a modular codebase and began transferring key functions from MATLAB to Python, enabling future extensions and wider accessibility. Although the fully automated extraction of flight data from Blackbox logs could not be completed, the knowledge gained provides an important basis for further investigation. Overall, the results show that offline

analysis and tuning of Betaflight FPV drones offers a robust, objective, and data-driven approach to optimize flight performance. The implemented methods enable reconstruction of control-loop dynamics from flight measurements, allowing detailed understanding of system behaviour and the impact of different controller settings. The work carried out so far lays a solid foundation for further development and refinement of the tool in our bachelor's thesis, where we will add new tuning capabilities, complete the Python migration, and validate through additional flight tests.

## 9 Directories

### 9.1 Bibliography

#### 9.1.1 Overview of Generative AI Systems

##### OpenAI (2025) – ChatGPT 5.1

- Available online: <https://chat.openai.com/>
- Functions:
  - Generation of text suggestions
  - Grammar and language correction
  - Support in writing LaTeX and MATLAB code

##### DeepL (2025) – DeepL Translator (Free Version)

- Available online: <https://www.deepl.com/translator>
- Functions:
  - Translation of text

## 9.2 List of Figures

|   |                                        |    |
|---|----------------------------------------|----|
| 1 | Chirp Signal example . . . . .         | 7  |
| 2 | Segmentations of Data . . . . .        | 11 |
| 3 | Gyro Control Loop . . . . .            | 11 |
| 4 | IFLIGHT aosmini build . . . . .        | 16 |
| 5 | Project Assignment . . . . .           | 30 |
| 6 | Project Schedule – Version 0 . . . . . | 33 |
| 7 | Project Schedule – Version 1 . . . . . | 33 |
| 8 | Project Schedule – Version 2 . . . . . | 34 |



## 10 Appendix

### 10.1 Project Management

#### 10.1.1 Project Assignment

aufgabenstellung.md

2025-09-15

## Projektarbeit

### Allgemein

#### Titel

Betaflight - Offline Controller Tuning

#### Beschreibung

Betaflight ist ein Open-Source Projekt, das sich auf die Regelung von FPV Quadcopter spezialisiert hat, dies insbesondere im Bereich von Race-Dronen. Die Firmware wird von einer grossen Community weiterentwickelt und ist auf vielen verschiedenen Hardware-Plattformen verfügbar. Betaflight ist die am häufigsten verwendete Firmware für FPV-Drohnen und wird von vielen Hobbyisten und Profis genutzt.

Für die Flugperformance ist die Qualität der Raten-Regelung von entscheidender Bedeutung. Die Regler sind dafür verantwortlich, dass die Drohne stabil fliegt, Störungen wie Wind und Strömungsabrisse (Prop-Wash) unterdrückt und möglichst agil auf Steuerbefehle reagiert. Die Regler und Filter müssen daher sorgfältig getunt und abgestimmt werden, um eine optimale Flugperformance zu erzielen. FPV Piloten fliegen üblicherweise im ACRO mode, sprich das System ist Ratengeregelt und die Stickinputs des Piloten werden als Sollwerte auf den Ratenregler gegeben. Das Tuning dieser Regler geschieht in der Community im Allgemeinen durch Trial and Error. Das bedeutet, dass die Piloten die Reglerparameter anpassen und dann testen. Dies kann jedoch zeitaufwendig und frustrierend sein, da es oft schwierig ist, insbesondere für Leien, die richtigen Parameter zu finden.

Bereits bestehend ist ein Proof of Concept, welches es ermöglicht die Raten-Regler offline basierend auf einem Messdaten-Modell der Drohne zu tunen (basierend auf Chirp-Signal Experiment während dem Flug). Das Projekt soll auf diesem Proof of Concept aufbauen und die Methode des Offline Tunings für die Raten-Regler (Gyro) weiterentwickeln. Ziel ist es, eine vollständige, einfach erweiterbare und sauber implementierte Open-Source Library zu entwickeln. Dies sowohl in MATLAB als auch in Python. Ausserdem soll für die Python Implementation eine ausführliche Dokumentation mit Beispielen für Anwender erstellt werden (Github Markdown Dokumentation). Das Ziel ist es, dass die Library von der Community genutzt werden kann. Dies soll dazu beitragen, die Flugperformance der Drohnen zu verbessern und den Piloten eine einfachere Möglichkeit zu bieten, ihre Drohnen zu tunen.

#### Studenten

Bianchi Yuri (biancyur)

Dort Janick (dortjan1)

Jurietti Dario (juriedar)

#### Betreuer

Hauptbetreuer: Michael Peter (pmic)

Nebenbetreuer: Prof. Dr. Ruprecht Altenburger (altb)

Camille Huber (hurc)

## Voraussetzungen

- GRT, RT1
- Zumindest eine Person ist im Besitz des Fernpiloten-Zeugnis A1/A3 (A2 optional jedoch empfohlen)
- Genügend hohe Haftpflichtversicherung, muss von den Studierenden selbst organisiert werden
- Erfahrung mit FPV Dronen (Bauen, Reparieren, Konfigurieren, Fliegen)
- MATLAB, Python

## Vereinbarung

Die Vereinbarung wird zwischen dem Betreuer Michael Peter (pmic) und den Studierenden Yuri Bianchi (biancyur), Janick Dort (dortjan1) und Dario Jurietti (juriedar).

Da es sich um eine dreier PA handelt wird das Projekt in drei separat bewertbare Teile aufteilt:

- MATLAB Implementation
- Python Implementation
- Markdown Dokumentation mit Beispielen der Matlab und Python Implementation

## Zeitplan

Siehe: <https://intra.zhaw.ch/departemente/school-of-engineering/bachelorstudium/projekt-und-bachelorarbeiten>

### Projektarbeit HS 25

|                              |                                                              |
|------------------------------|--------------------------------------------------------------|
| Ab 15.09.2025                | Ausgabe der Aufgabenstellung durch Dozierende an Studierende |
| Bis 19.12.2025 bis 18:00 Uhr | Abgabe der Projektarbeit in Complexis                        |
| 03.02.2026                   | Prov. Notenfreischaltung für Studierende ab 18:00 Uhr        |
| 27.02.2026                   | Def. Notenfreischaltung für Studierende ab 18:00 Uhr         |

## Arbeitsplatz

TE 617

## Hardware

- Hardware kann von Michael Peter und IMS bezogen werden
  - 2 x 3.5 Zoll Dronen
  - 2 x 5.0 Zoll Dronen
- Es können auch noch zwei weitere Dronen (1 x 3.5 Zoll, 1 x 5.0 Zoll) gebaut werden

## Ergebnisdarstellung

Ein Bericht im Umfang von 20-30 Seiten dokumentiert den Entwicklungsweg geeignet. Die Software wird in einem öffentlichen GitHub Repository abgelegt. Des weiteren soll eine Dokumentation mit Beispielen in Markdown erstellt werden. Die technische Dokumentation ist ebenso in Markdown zu verfassen.

## Fachgebiet

Primäres Fachgebiet

Regelungstechnik und Advanced Control

Weitere Fachgebiete

- Datenvisualisierung
- Digitale Signalverarbeitung
- Software Engineering

## Verschiedenes

Studiengang

Systemtechnik

Industriepartner

Keiner

Institut / Zentren

Institut für Mechatronische Systeme (IMS)

Interne Partner

Keine

## Informationslink

[https://github.com/pichim/bf\\_controller\\_tuning](https://github.com/pichim/bf_controller_tuning)

Bemerkungen

- Grundsätzlich ist die Idee die PA aufbauend in einer BA zu erweitern

Sprache

Deutsch / Englisch

## Studenten Detail

- Yuri Bianchi (biancyur) - Automatiker - Leader, Pilot, Regelungstechnik & Signalverarbeitung
- Dario Jurietti (juriedar) - Biolaborant - Datenverarbeitung, Software
- Janick Dort (dortjan1) - Elektroplaner - Datenverarbeitung, Pilot, Regelungstechnik & Signalverarbeitung

Die Studenten haben schon PM 1-4 zusammen absolviert und sind ein eingespieltes Team.



### 10.1.2 Project Schedule – Version 0

This project schedule was created at the beginning of the project.

Zeitplan PA Drone Tuning

Semesterwoche

| Week                                           | Start SW | Ende SW | 1     | 2     | 3     | 4    | 5     | 6     | 7     | 8    | 9     | 10    | 11    | 12   | 13   | 14    |
|------------------------------------------------|----------|---------|-------|-------|-------|------|-------|-------|-------|------|-------|-------|-------|------|------|-------|
| Date                                           |          |         | 15.09 | 22.09 | 29.09 | 6.10 | 13.10 | 20.10 | 27.10 | 3.11 | 10.11 | 17.11 | 24.11 | 1.12 | 8.12 | 15.12 |
| Description                                    |          |         |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Erstellung Zeitplan                            | 1        | 1       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Testversuche Programm und Drohne               | 1        | 3       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Simulation Regler (Simulink/MATLAB)            | 1        | 3       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Codestruktur definieren                        | 2        | 4       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Drohne bauen                                   | 3        | 5       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Milestone Meeting 1                            | 5        | 5       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| MATLAB nach Codestruktur (Klassen) umschreiben | 4        | 7       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Weiterentwicklung MATLAB Code                  | 7        | 9       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| .txt Logfile in .csv umwandeln                 | 9        | 11      |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Milestone Meeting 2                            | 9        | 9       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| MATLAB -> Python                               | 9        | 12      |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Optimierung Chirp Signal Betaflight            | 12       | 14      |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
|                                                |          |         |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Doku                                           | 10       | 14      |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| GitHub Anleitung                               | 10       | 14      |       |       |       |      |       |       |       |      |       |       |       |      |      |       |

Abbildung 6: Project Schedule – Version 0

### 10.1.3 Project Schedule – Version 1

This project schedule was created prior to the first milestone meeting and approved during the meeting.

Zeitplan PA Drone Tuning

Semesterwoche

| Week                                           | Start SW | Ende SW | 1     | 2     | 3     | 4    | 5     | 6     | 7     | 8    | 9     | 10    | 11    | 12   | 13   | 14    |
|------------------------------------------------|----------|---------|-------|-------|-------|------|-------|-------|-------|------|-------|-------|-------|------|------|-------|
| Date                                           |          |         | 15.09 | 22.09 | 29.09 | 6.10 | 13.10 | 20.10 | 27.10 | 3.11 | 10.11 | 17.11 | 24.11 | 1.12 | 8.12 | 15.12 |
| Description                                    |          |         |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Erstellung Zeitplan                            | 1        | 1       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Testversuche Programm und Drohne               | 1        | 4       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Simulation Regler (Simulink/MATLAB)            | 1        | 3       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Simulation Übertragungsfunktion                | 2        | 4       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Codestruktur definieren                        | 2        | 4       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Simulation PID Regler                          | 3        | 4       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Simulation Spektrum                            | 4        | 5       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Drohne bauen                                   | 5        | 7       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Milestone Meeting 1                            | 5        | 5       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Simulation Spektrogramm                        | 5        | 6       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| MATLAB nach Codestruktur (Klassen) umschreiben | 5        | 8       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Weiterentwicklung MATLAB Code                  | 7        | 9       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| .txt Logfile in .csv umwandeln                 | 1        | 6       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Milestone Meeting 2                            | 9        | 9       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| MATLAB -> Python                               | 8        | 10      |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Weiterentwicklung Python Code                  | 10       | 13      |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Optimierung Chirp Signal Betaflight            | 12       | 14      |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
|                                                |          |         |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Doku                                           | 10       | 14      |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| GitHub Anleitung                               | 10       | 14      |       |       |       |      |       |       |       |      |       |       |       |      |      |       |

Abbildung 7: Project Schedule – Version 1

### 10.1.4 Project Schedule – Version 2

This project schedule was created prior to the second milestone meeting and approved during the meeting.

**Zeitplan PA Drone Tuning**

Semesterwoche

| Week                                                       | Start SW | Ende SW | 1     | 2     | 3     | 4    | 5     | 6     | 7     | 8    | 9     | 10    | 11    | 12   | 13   | 14    |
|------------------------------------------------------------|----------|---------|-------|-------|-------|------|-------|-------|-------|------|-------|-------|-------|------|------|-------|
| Date                                                       |          |         | 15.09 | 22.09 | 29.09 | 6.10 | 13.10 | 20.10 | 27.10 | 3.11 | 10.11 | 17.11 | 24.11 | 1.12 | 8.12 | 15.12 |
| <b>Description</b>                                         |          |         |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Erstellung Zeitplan                                        | 1        | 1       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Testversuche Programm und Drohne                           | 1        | 4       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Simulation Regler (Simulink/MATLAB)                        | 1        | 3       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Simulation Übertragungsfunktion                            | 2        | 4       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Codestruktur definieren                                    | 2        | 4       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Simulation PID Regler                                      | 3        | 4       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Simulation Spektrum                                        | 4        | 5       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Drohne bauen                                               | 5        | 12      |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Milestone Meeting 1                                        | 5        | 5       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Simulation Spektrogramm                                    | 5        | 6       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| MATLAB nach Codestruktur (Klassen) umschreiben             | 5        | 8       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Weiterentwicklung MATLAB-Code                              | 7        | 9       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| .txt Logfile in .csv umwandeln                             | 1        | 6       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Milestone Meeting 2                                        | 9        | 9       |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Erstellung eines Miniskript in Python                      | 7        | 10      |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| GitHub Dokumentation                                       | 8        | 11      |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Optimierung Codestruktur MATLAB                            | 9        | 11      |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Umschreiben der MATLAB Library in Python mit Funktionstest | 10       | 13      |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| MATLAB → Python                                            | 8        | 10      |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Weiterentwicklung Python-Code                              | 10       | 13      |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Optimierung Chirp-Signal-Betaflight                        | 12       | 14      |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| Doku                                                       | 10       | 14      |       |       |       |      |       |       |       |      |       |       |       |      |      |       |
| GitHub Anleitung                                           | 10       | 14      |       |       |       |      |       |       |       |      |       |       |       |      |      |       |

Abbildung 8: Project Schedule – Version 2

## 10.2 Additional Information